1600 Liinux USB 摄像头开发记录

前期准备

- **make menuconfig**
  - BR2_PACKAGE_LIBV4L_UTILS
  - BR2_PACKAGE_LIBJPEG
- **make linux-menuconfig**
  - MEDIA_CAMERA_SUPPORT
  - CONFIG_MEDIA_USB_SUPPORT
  - CONFIG_USB_VIDEO_CLASS
- **添加v4l2grab测试源码到 SOURCE/hc-examples**
  - 具体源码请见本文下面章节
- **修改 SOURCE/hc-examples/meson.build**
  - executable('v4l2grab', v4l2grab, c_args: cflags_v4l2grab, install : false)
  - 将这句的注释重新打开，并将最后的 install由 false改为true
- **检查patches目录，查看下述的patch包是否存在**
  - patches/linux-4.4.186/0022-fix-musb-isochronus-xfer-bug.patch

完成上述之后，执行`make all;make hc-examples-rebuild linux-rebuild all`即可

测试demo示例

github 的v4l2grab

https://github.com/twam/v4l2grab

```
# Install the required libraries (libv4l and libjpeg) and autotools
# Ubuntu/Debian:
$ sudo apt-get install libjpeg-dev libv4l-dev autoconf automake libtool
# Gentoo:
$ emerge -va libjpeg-turbo libv4l

# Clone the repository
$ git clone https://github.com/twam/v4l2grab.git
$ git clone https://ghproxy.com/https://github.com/twam/v4l2grab.git

# Go into directory
$ cd v4l2grab

# Creating autotools files
$ ./autogen.sh

# Run configure
$ ./configure

```

```
# Run make
$ make
$ sudo make install
```

简化的v4l2grab

具体测试demo源码(v4l2grab)请见本文下面的章节

板端上电进入命令行后，如下操作

```
$ ls /dev/video0     ## 如果插入USB摄像头后,请检查/dev目录下是否已经生成如下设备文件,若有则表示已经成功识别
$ v4l2grab           ## 此时就会在/tmp目录生成两个图片文件
```

patch补丁说明

patches/linux-4.4.186/0022-fix-musb-isochronus-xfer-bug.patch

在linux-4.4.186版本中的musb驱动有个bug, 它是默认要求musb驱动的ISO传输必须支持`high bandwidth`模式. 这个模式意思就是说ISO传输用的EP的FIFO要有足够大, 这样就能一次过把ISO传输的EP最大包长一笔过传输完. 但hichip 1600 MUSB的EP FIFO仅有1KB深度, 部分摄像头的ISO EP传输最大包长可达到5KB.

测试程序v4l2grab源码

yuv.h

```
#ifndef _YUV_H_
#define _YUV_H_
void YUV420toYUV444(int width, int height, unsigned char* src, unsigned char* dst);
#endif

```

config.h

```
/* config.h.  Generated from config.h.in by configure.  */
/* config.h.in.  Generated from configure.ac by autoheader.  */

/* Define to 1 if you have the `clock_gettime' function. */
#define HAVE_CLOCK_GETTIME 1

/* Define to 1 if you have the <fcntl.h> header file. */
#define HAVE_FCNTL_H 1

/* Define to 1 if you have the `getpagesize' function. */
```

```
#define HAVE_GETPAGESIZE 1

/* Define to 1 if you have the <inttypes.h> header file. */
#define HAVE_INTTYPES_H 1

/* Define to 1 if you have the `jpeg' library (-ljpeg). */
#define HAVE_LIBJPEG 1

/* Define to 1 if you have the `v4l2' library (-lv4l2). */
#define HAVE_LIBV4L2 1

/* Define to 1 if your system has a GNU libc compatible `malloc' function,
and
   to 0 otherwise. */
#define HAVE_MALLOC 1

/* Define to 1 if you have the <malloc.h> header file. */
#define HAVE_MALLOC_H 1

/* Define to 1 if you have the <memory.h> header file. */
#define HAVE_MEMORY_H 1

/* Define to 1 if you have the `select' function. */
#define HAVE_SELECT 1

/* Define to 1 if you have the <stdint.h> header file. */
#define HAVE_STDINT_H 1

/* Define to 1 if you have the <stdlib.h> header file. */
#define HAVE_STDLIB_H 1

/* Define to 1 if you have the `strerror' function. */
#define HAVE_STRERROR 1

/* Define to 1 if you have the <strings.h> header file. */
#define HAVE_STRINGS_H 1

/* Define to 1 if you have the <string.h> header file. */
#define HAVE_STRING_H 1

/* Define to 1 if you have the <sys/ioctl.h> header file. */
#define HAVE_SYS_IOCTL_H 1

/* Define to 1 if you have the <sys/stat.h> header file. */
#define HAVE_SYS_STAT_H 1

/* Define to 1 if you have the <sys/time.h> header file. */
#define HAVE_SYS_TIME_H 1

/* Define to 1 if you have the <sys/types.h> header file. */
#define HAVE_SYS_TYPES_H 1

/* Define to 1 if you have the <unistd.h> header file. */
#define HAVE_UNISTD_H 1

/* Name of package */
```

```
#define PACKAGE "v4l2grab"

/* Define to the address where bug reports for this package should be sent.
*/
#define PACKAGE_BUGREPORT "Tobias_Mueller@twam.info"

/* Define to the full name of this package. */
#define PACKAGE_NAME "v4l2grab"

/* Define to the full name and version of this package. */
#define PACKAGE_STRING "v4l2grab 0.4.1-19-gf8d8844-dirty"

/* Define to the one symbol short name of this package. */
#define PACKAGE_TARNAME "v4l2grab"

/* Define to the home page for this package. */
#define PACKAGE_URL ""

/* Define to the version of this package. */
#define PACKAGE_VERSION "0.4.1-19-gf8d8844-dirty"

/* Define to 1 if you have the ANSI C header files. */
#define STDC_HEADERS 1

/* Version number of package */
#define VERSION "0.4.1-19-gf8d8844-dirty"

/* Define to rpl_malloc if the replacement function should be used. */
/* #undef malloc */

/* Define to `unsigned int' if <sys/types.h> does not define. */
/* #undef size_t */

```

v4l2grab.c

```
/*****************************************************************************
*
 *   v4l2grab Version 0.3
*
 *   Copyright (C) 2012 by Tobias Müller
*
 *   Tobias_Mueller@twam.info
*
 *
*
 *   based on V4L2 Specification, Appendix B: Video Capture Example
*
 *   (http://v4l2spec.bytesex.org/spec/capture-example.html)
*
 *
*
```

```
 *   This program is free software; you can redistribute it and/or modify
*
 *   it under the terms of the GNU General Public License as published by
*
 *   the Free Software Foundation; either version 2 of the License, or
*
 *   (at your option) any later version.
*
 *
*
 *   This program is distributed in the hope that it will be useful,
*
 *   but WITHOUT ANY WARRANTY; without even the implied warranty of
*
 *   MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
*
 *   GNU General Public License for more details.
*
 *
*
 *   You should have received a copy of the GNU General Public License
*
 *   along with this program; if not, write to the
*
 *   Free Software Foundation, Inc.,
*
 *   59 Temple Place - Suite 330, Boston, MA  02111-1307, USA.
*

****************************************************************************
/


/****************************************************************************
 *   Modification History
*
 *
*
 *   Matthew Witherwax      21AUG2013
*
 *      Added ability to change frame interval (ie. frame rate/fps)
*
 * Martin Savc              7JUL2015
 *      Added support for continuous capture using SIGINT to stop.

****************************************************************************
/

// compile with all three access methods
#if !defined(IO_READ) && !defined(IO_MMAP) && !defined(IO_USERPTR)
#define IO_READ
#define IO_MMAP
#define IO_USERPTR
#endif

#include <stdio.h>
```

```
#include <stdlib.h>
#include <string.h>
#include <assert.h>
#include <getopt.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>
#include <malloc.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <sys/time.h>
#include <time.h>
#include <sys/mman.h>
#include <sys/ioctl.h>
#include <asm/types.h>
#include <linux/videodev2.h>
#include <jpeglib.h>
#include <libv4l2.h>
#include <signal.h>
#include <stdint.h>
#include <inttypes.h>
#include <poll.h>

#include "config.h"
#include "yuv.h"

#define CLEAR(x) memset (&(x), 0, sizeof (x))

#ifndef VERSION
#define VERSION "unknown"
#endif

#if defined(IO_MMAP) || defined(IO_USERPTR)
// minimum number of buffers to request in VIDIOC_REQBUFS call
#define VIDIOC_REQBUFS_COUNT 2
#endif

typedef enum {
#ifdef IO_READ
        IO_METHOD_READ,
#endif
#ifdef IO_MMAP
        IO_METHOD_MMAP,
#endif
#ifdef IO_USERPTR
        IO_METHOD_USERPTR,
#endif
} io_method;

typedef struct tagBITMAPFILEHEADER{
    unsigned short    bfType;                // the flag of bmp, value is
"BM"
    unsigned int    bfSize;                // size BMP file ,unit is bytes
    unsigned int     bfReserved;            // 0
    unsigned int     bfOffBits;             // must be 54

```

```
}BITMAPFILEHEADER;

typedef struct tagBITMAPINFOHEADER{
    unsigned int     biSize;                // must be 0x28
    unsigned int     biWidth;           //
    unsigned int     biHeight;          //
    unsigned short       biPlanes;          // must be 1
    unsigned short       biBitCount;            //
    unsigned int     biCompression;         //
    unsigned int     biSizeImage;       //
    unsigned int     biXPelsPerMeter;   //
    unsigned int     biYPelsPerMeter;   //
    unsigned int     biClrUsed;             //
    unsigned int     biClrImportant;        //
}BITMAPINFOHEADER;

struct videobuffer {
        void *                  start;
        size_t                  length;
};

//#define  IMAGEWIDTH    1280
//#define  IMAGEHEIGHT   720
#define VIDEOCOUNT  3
#define BMP          "/tmp/image_rgb.bmp"
#define JPEG          "/tmp/image_jpeg.jpg"
#define YUV            "/tmp/image_yuv.yuv"

static struct videobuffer framebuf[VIDEOCOUNT];
//static   struct   v4l2_capability   cap;//device func,
VIDIOC_QUERYCAP=cmd
//static struct v4l2_fmtdesc fmtdesc;//video format

//static io_method        io              = IO_METHOD_MMAP;
static unsigned char* starter;
static unsigned char* newBuf;
static int  fd_video            = -1;
//struct buffer *         buffers         = NULL;
//static unsigned int     n_buffers       = 0;

// global settings
static unsigned int width = 1280;//640;
static unsigned int height = 720;//480;

/**
SIGINT interput handler
*/
char save_bmp_file(unsigned char *rgb_date, int date_len);

int yuyv_2_rgb888(const void *p, int size, unsigned char *frame_buffer)
 {
    int           i,j;
    unsigned char y1,y2,u,v;
    int r1,g1,b1,r2,g2,b2;
    char * pointer;

```

```
    pointer = p;
    for(i=0;i<480;i++)
    {
        for(j=0;j<320;j++)
        {
            y1 = *( pointer + (i*320+j)*4);
            u  = *( pointer + (i*320+j)*4 + 1);
            y2 = *( pointer + (i*320+j)*4 + 2);
            v  = *( pointer + (i*320+j)*4 + 3);

            r1 = y1 + 1.042*(v-128);
            g1 = y1 - 0.34414*(u-128) - 0.71414*(v-128);
            b1 = y1 + 1.772*(u-128);
            r2 = y2 + 1.042*(v-128);

            g2 = y2 - 0.34414*(u-128) - 0.71414*(v-128);
            b2 = y2 + 1.772*(u-128);

            if(r1>255)                r1 = 255;
            else if(r1<0)                r1 = 0;
            if(b1>255)                b1 = 255;
            else if(b1<0)                b1 = 0;
            if(g1>255)                g1 = 255;
            else if(g1<0)                g1 = 0;
            if(r2>255)                r2 = 255;
            else if(r2<0)                r2 = 0;
            if(b2>255)                b2 = 255;
            else if(b2<0)                b2 = 0;
            if(g2>255)                g2 = 255;
            else if(g2<0)                g2 = 0;

            *(frame_buffer + ((480-1-i)*320+j)*6    ) = (unsigned char)b1;
            *(frame_buffer + ((480-1-i)*320+j)*6 + 1) = (unsigned char)g1;
            *(frame_buffer + ((480-1-i)*320+j)*6 + 2) = (unsigned char)r1;
            *(frame_buffer + ((480-1-i)*320+j)*6 + 3) = (unsigned char)b2;
            *(frame_buffer + ((480-1-i)*320+j)*6 + 4) = (unsigned char)g2;
            *(frame_buffer + ((480-1-i)*320+j)*6 + 5) = (unsigned char)r2;
        }
    }
    printf("change to RGB OK \n");
}

char save_bmp_file(unsigned char *rgb_date, int date_len)
{
    FILE * fp1;
    BITMAPFILEHEADER   bf;
    BITMAPINFOHEADER   bi;
    unsigned char bf_header[16]={0};

    printf("%s ####L%d\n", __func__, __LINE__);
    //Set BITMAPINFOHEADER
    bi.biSize = 40;
    bi.biWidth = width;
    bi.biHeight = height;
    bi.biPlanes = 1;
    bi.biBitCount = 24;
```

```
    bi.biCompression = 0;
    bi.biSizeImage = bi.biWidth * bi.biHeight * 3;
    bi.biXPelsPerMeter = 0;
    bi.biYPelsPerMeter = 0;
    bi.biClrUsed = 0;
    bi.biClrImportant = 0;

    //Set BITMAPFILEHEADER
    bf.bfType = 0x4d42;
    bf.bfSize = 54 + bi.biSizeImage;
    bf.bfReserved = 0;
    bf.bfOffBits = 54;
    bf_header[0] = bf.bfType & 0xFF;
    bf_header[1] = (bf.bfType >> 8) & 0xFF;
    bf_header[2] = bf.bfSize  & 0xFF;
    bf_header[3] = (bf.bfSize >>8) & 0xFF;
    bf_header[4] = (bf.bfSize >> 16) & 0xFF;
    bf_header[5] = (bf.bfSize >>24) & 0xFF;
    bf_header[10] = bf.bfOffBits  & 0xFF;
    bf_header[11] = (bf.bfOffBits >>8) & 0xFF;
    bf_header[12] = (bf.bfOffBits >> 16) & 0xFF;
    bf_header[13] = (bf.bfOffBits >>24) & 0xFF;

    fp1 = fopen(BMP, "wb");
    if(!fp1)
    {
        printf("open 'BMP' error\n");
        return(FALSE);
    }
    fwrite(bf_header, 14, 1, fp1);
    fwrite(&bi, 40, 1, fp1);

    fwrite(rgb_date, bi.biSizeImage, 1, fp1);
    printf("save 'BMP' OK\n");

    fclose(fp1);

}

int openCamera(int id)
{
    char devicename[12];
    sprintf(devicename,"/dev/video%d",id);
    fd_video = open(devicename, O_RDWR | O_NONBLOCK, 0);
    if(fd_video <0 ){
        printf("open video%d fail.\n", id);
        return -1;
    }

    /*fd_fb = open("/dev/fb0", O_RDWR | O_NONBLOCK, 0);
    if(fd_fb <0 ){
        printf("open screen fail.\n");
        return -1;
    }*/
    return 0;
}
```

```
264
265 void capabilityCamera(void)
266 {
267     struct v4l2_capability cap;
268     //query cap
269     if (ioctl(fd_video, VIDIOC_QUERYCAP, &cap) == -1)
270     {
271         printf("Error opening device: unable to query device.\n");
272         return (FALSE);
273     }
274     else
275     {
276         printf("driver:\t\t%s\n",cap.driver);
277         printf("card:\t\t%s\n",cap.card);
278         printf("bus_info:\t%s\n",cap.bus_info);
279         printf("version:\t%d\n",cap.version);
280         printf("capabilities:\t%x\n",cap.capabilities);
281         if ((cap.capabilities & V4L2_CAP_VIDEO_CAPTURE) ==
    V4L2_CAP_VIDEO_CAPTURE)
282         {
283             printf("Device: supports capture.\n");
284         }
285
286         if ((cap.capabilities & V4L2_CAP_STREAMING) == V4L2_CAP_STREAMING)
287         {
288             printf("Device: supports streaming.\n");
289         }
290         //printf("device_caps 0x%x\n",cap.device_caps);
291     }
292 }
293
294 void enumfmtCamera(void)
295 {
296     int ret;
297     int i;
298     v4l2_std_id std_id;
299     static struct v4l2_fmtdesc fmtdesc;//video format
300
301     do{
302         ret = ioctl(fd_video, VIDIOC_QUERYSTD,&std_id);
303     }while (ret == -1 && errno == EAGAIN);
304     switch(std_id){
305         case V4L2_STD_NTSC:
306             printf("STD NTSC format:\n");
307             break;
308         case V4L2_STD_PAL:
309             printf("STD PAL format:\n");
310             break;
311         default:
312             printf("==STD no define format=0x%x:\n", std_id);
313             break;
314     }
315
316     memset(&fmtdesc, 0, sizeof(fmtdesc));
317     fmtdesc.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
318     printf("-------------VIDIOC_ENUM_FMT-------------\n");
```

```
    while((ioctl(fd_video, VIDIOC_ENUM_FMT, &fmtdesc)) != -1)
    {
        printf("index:%d   \npixelformat:%c%c%c%c
\ndescription:%s\n",fmtdesc.index, fmtdesc.pixelformat&0xff,
            (fmtdesc.pixelformat>>8)&0xff,(fmtdesc.pixelformat>>16)&0xff,
            (fmtdesc.pixelformat>>24)&0xff,fmtdesc.description);
        fmtdesc.index++;
    }
}

/* 4??????? VIDIOC_S_FMT struct v4l2_format */
int setfmtCamera(void)
{
    int ret;
    struct v4l2_format format;

    format.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
    format.fmt.pix.width = width;
    format.fmt.pix.height = height;
    format.fmt.pix.pixelformat = V4L2_PIX_FMT_MJPEG;//V4L2_PIX_FMT_YUYV;
// ???yuyv????
    format.fmt.pix.field = V4L2_FIELD_INTERLACED;
    ret = ioctl(fd_video, VIDIOC_S_FMT, &format);
    if(ret < 0){
        printf("VIDIOC_S_FMT fail. ret=0x%x\n", ret);
        return -1;
    }
    memset(&format, 0, sizeof(format));
    format.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
    ret = ioctl(fd_video, VIDIOC_G_FMT, &format);
    if(ret < 0)
    {
        printf("VIDIOC_G_FMT fail. ret=0x%x\n", ret);
        return -1;
    }
    else
    {
        printf("fmt.type:\t\t%d\n",format.type);
        printf("pix.pixelformat:\t%c%c%c%c\n",format.fmt.pix.pixelformat &
0xFF, (format.fmt.pix.pixelformat >> 8) & 0xFF,
            (format.fmt.pix.pixelformat >> 16) & 0xFF,
(format.fmt.pix.pixelformat >> 24) & 0xFF);
        printf("pix.width:\t\t%d\n",format.fmt.pix.width);
        printf("pix.height:\t\t%d\n",format.fmt.pix.height);
        printf("pix.field:\t\t%d\n",format.fmt.pix.field);
    }
    return 0;
}

int setvideofps(void)
{
    int ret;
    struct v4l2_streamparm setfps;
    //set fps
    memset(&setfps, 0, sizeof(struct v4l2_streamparm));
    setfps.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
```

```
371     setfps.parm.capture.timeperframe.numerator = 1;//10;
372     setfps.parm.capture.timeperframe.denominator = 10;//10;
373     ret = ioctl(fd_video, VIDIOC_S_PARM, &setfps);
374     if (ret == -1){
375         printf("init VIDIOC_S_PARM Err!\n");
376     }
377     else
378         printf("Set fps OK!ret=%d\n", ret);
379 
380     ret = ioctl(fd_video, VIDIOC_G_PARM, &setfps);
381     if (ret == -1){
382         printf("init VIDIOC_G_PARM Err!\n");
383     }
384     else
385         printf("Get fps OK!ret=%d\n", ret);
386     printf("Set uvc frame bps: %d / %d\t[OK], type=%d\n",
setfps.parm.capture.timeperframe.denominator,
387         setfps.parm.capture.timeperframe.numerator, setfps.type );
388     return 0;
389 }
390 /* 5??????????VIDIOC_REQBUFS struct v4l2_requestbuffers,
391 *  ?????????????? VIDIOC_QUERYBUF struct v4l2_buffer mmap,??????????
392 VIDIOC_QBUF
393 */
394 int initmmap(void)
395 {
396     struct v4l2_requestbuffers reqbuf;
397     int i, ret;
398 
399     /*pfb = (char*)mmap(NULL, IMAGEWIDTH*IMAGEHEIGHT*4,
PROT_WRITE|PROT_READ,MAP_SHARED, fd_fb, 0);
400     if (pfb == NULL){
401         printf("mmap pfb Error!!\n");
402         return FALSE;
403     }*/
404     //request for 4 buffers
405     reqbuf.count = VIDEOCOUNT;
406     reqbuf.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
407     reqbuf.memory = V4L2_MEMORY_MMAP;
408     ret = ioctl(fd_video, VIDIOC_REQBUFS, &reqbuf);
409     if(-1 == ret){
410         printf("VIDIOC_REQBUFS fail. ret=0x%x\n", ret);
411         return -1;
412     }
413     if (reqbuf.count < 2)
414         printf("VIDIOC_REQBUFS count=%d Err!!\n", reqbuf.count);
415 
416     //v4l2_buffer
417     printf("----------------mmap---------------\n");
418     for(i =0; i < reqbuf.count; i++){
419         struct v4l2_buffer buf;
420         memset(&buf, 0, sizeof(buf));
421         buf.index = i;
422         buf.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
423         buf.memory = V4L2_MEMORY_MMAP;
424         ret = ioctl(fd_video, VIDIOC_QUERYBUF, &buf);
```

```
        if (-1 == ret)
        {
            printf("query buffer error. ret=0x%x\n", ret);
            return(FALSE);
        }
        framebuf[i].length = buf.length;
        framebuf[i].start = mmap(NULL, buf.length, PROT_READ|PROT_WRITE,
            MAP_SHARED, fd_video, buf.m.offset);
        if(framebuf[i].start == MAP_FAILED){
            printf("buffer map error\n");
            return -1;
        }
        printf("start:%x  length:%d\n",(unsigned int)framebuf[i].start,
framebuf[i].length);
    }
    return 0;
}


/* 6?????????
 * ??????? VIDIOC_QBUF  struct v4l2_buffer
 * ????? VIDIOC_STREAMON
 */
static int startcap(void)
{
    int ret = -1, i = 0;
    struct v4l2_buffer buf;
    enum v4l2_buf_type type;

    //queue
    for(i=0;i < VIDEOCOUNT; i++){
        memset(&buf, 0, sizeof(buf));
        buf.index = i;
        buf.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
        buf.memory = V4L2_MEMORY_MMAP;
        ret = ioctl(fd_video, VIDIOC_QBUF, &buf);
        if(0 != ret){
            printf("VIDIOC_QBUF fail. ret=0x%x\n", ret);
            return -1;
        }
    }
    //signal(SIGINT mysignal);//interrrupt

    type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
    ret = ioctl(fd_video, VIDIOC_STREAMON, &type);
    if(0 != ret)
        printf("VIDIOC_STREAMON fail.ret=0x%x\n", ret);

    return 0;
}
/* 6?????????? ??poll??
 *  ?????????? VIDIOC_DQBUF
 */

static int readfram(void)
{
```

```
480     struct pollfd pollfd;
481     int ret,i;
482     char filename[50];
483     struct v4l2_buffer buf;
484 
485     //check had read frame or not, use poll fun
486     printf("Check Get data:\n");
487     for(i =0; i<20 ;i++){
488         memset(&pollfd, 0, sizeof(pollfd));
489         pollfd.fd = fd_video;
490         pollfd.events = POLLIN;
491         ret = poll(&pollfd, 1, 800);
492         if(-1 == ret){
493             printf("VIDIOC_QBUF fail. ret=0x%x\n", ret);
494             return -1;
495         }else if(0 == ret){
496             printf("poll time out\n");
497             continue;
498         }
499         printf("-poll revents=0x%x-\n", pollfd.revents);
500         // static struct v4l2_buffer buf;
501 
502         if(pollfd.revents & POLLIN){
503             printf("start get----\n");
504             memset(&buf, 0, sizeof(buf));
505             buf.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
506             buf.memory = V4L2_MEMORY_MMAP;
507             ret = ioctl(fd_video, VIDIOC_DQBUF, &buf);
508             if(0 != ret){
509                 printf("VIDIOC_DQBUF fail.ret=0x%x\n", ret);
510                 return -1;
511             }
512             //v4l2_buffer
513             FILE *file = fopen(YUV, "wb");
514             ret = fwrite((char*)framebuf[buf.index].start, 1, buf.length,
file);
515             fclose(file);
516 
517             #if 1// RGB????
518             starter = (unsigned char*)framebuf[buf.index].start;
519             newBuf = (unsigned char*)calloc((unsigned int)
(framebuf[buf.index].length*3/2), 1);
520             yuyv_2_rgb888(starter, framebuf[buf.index].length*3/2, newBuf);
521             save_bmp_file(newBuf, framebuf[buf.index].length*3/2);
522             #endif
523             ret = ioctl(fd_video, VIDIOC_QBUF, &buf);
524             if(0 != ret){
525                 printf("VIDIOC_QBUF fail. ret=0x%x\n", ret);
526                 return -1;
527             }
528         }
529     }
530     return ret;
531 }
532 
533 /* ???????????? */
```

```
static void closeCamera(void)
{
    int ret=-1, i;
    enum v4l2_buf_type type;

    printf("%s ####L%d\n", __func__, __LINE__);
    type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
    ret = ioctl(fd_video,VIDIOC_STREAMOFF, &type);
    if(0 != ret){
        printf("VIDIOC_QBUF fail. ret=0x%x\n", ret);
        return ;
    }
    for(i = 0; i < VIDEOCOUNT; i++){
        munmap(framebuf[i].start, framebuf[i].length);
    }
    close(fd_video);
    fd_video=NULL;
    //munmap(pfb, IMAGEWIDTH*IMAGEHEIGHT*4);
    //pfb = NULL;
    //close(fd_fb);
    //fd_fb=NULL;
}

int main(int argc, char **argv)
{
    // open and initialize device
    //init_v4l2();
    //v4l2_grab();

    // start capturing
    //close_v4l2(1);
    openCamera(0);

    capabilityCamera();
    enumfmtCamera();
    setfmtCamera();
    setvideofps();
    initmmap();
    startcap();
    readfram();
    closeCamera();
    return 0;
}

```